

CSA1711 - Artificial Intelligence

CO5 Assessment Tool - 3

Reverse Engineering - Answer Script

1. Reverse Engineering a Recommendation System

Scenario: A large-scale e-commerce platform provides highly accurate product recommendations using real-time data. Analyze the system and reverse engineer its architecture using PySpark and RDD operations. Identify how data is collected, processed, and transformed, and infer the role of intelligent agents in updating recommendations dynamically.

Reverse-Engineered Architecture

Observed behaviour: recommendations change within moments of a click, purchase, or even just browsing a new category, and different interaction types (view/click/purchase) clearly carry different weight in what gets recommended next. Working backward from this behaviour, the most plausible internal pipeline is:

- **Data Collection:** a clickstream/event-capture layer logs every view, click, and purchase as an immutable event (user_id, item_id, event_type, price, timestamp), published onto a streaming bus (e.g., Kafka) partitioned by user or item.
- **Data Processing (PySpark / RDD):** Spark Structured Streaming (or micro-batch Spark jobs) consumes these events into RDDs/DataFrames; a map step assigns an implicit-feedback weight per event type, and a reduceByKey aggregates weights per (user, item) pair — this recreates an 'engagement score' feature without ever needing an explicit star rating.
- **Data Transformation:** the aggregated engagement scores feed a collaborative-filtering stage (e.g., Spark MLlib ALS) that learns latent user/item factors, refreshed on a rolling batch schedule, while the most recent events are scored incrementally between full retrains.
- **Intelligent Agent Role:** a Utility-Based Agent sits at the serving layer — for each user it scores every candidate item by combining relevance (from the collaborative-filtering factors), price-fit, and novelty into a single utility value, then ranks and serves the top-N. Because this scoring re-runs on each new event, recommendations update dynamically without waiting for a full model retrain.

Implementation (PySpark / RDD)

system1_recommendation.py

```
"""
System 1 - Reverse-Engineered Recommendation System (E-Commerce)
-----
Reconstructs, at small scale, the pipeline a large e-commerce platform likely
runs: raw clickstream/purchase events -> feature extraction (RDD transformations)
-> per-user item scoring (Utility-Based Agent) -> ranked, dynamically updated
recommendations.

Run: python3 system1_recommendation.py
"""
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReverseEng-RecommendationSystem").master("local[2]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# ---- Step 1: Raw event stream (in production this arrives via Kafka -> Spark Structured Streaming) ----
# (user_id, item_id, event_type, price, rating_if_purchased)
raw_events = [
    ("U1", "I100", "view", 25.0, None),
    ("U1", "I100", "click", 25.0, None),
    ("U1", "I101", "purchase", 40.0, 5),
    ("U1", "I102", "view", 15.0, None),
    ("U2", "I100", "purchase", 25.0, 4),
```

```

    ("U2", "I103", "view", 60.0, None),
    ("U2", "I103", "click", 60.0, None),
    ("U3", "I101", "view", 40.0, None),
    ("U3", "I104", "purchase", 20.0, 3),
    ("U1", "I104", "click", 20.0, None),
]
events_rdd = sc.parallelize(raw_events)

# ---- Step 2: Feature extraction via RDD transformations ----
# Weight each interaction type (implicit feedback strength) - this recreates the
# "engagement scoring" step a recommender's feature pipeline performs.
WEIGHTS = {"view": 1.0, "click": 2.0, "purchase": 5.0}

def to_weighted(rec):
    user, item, etype, price, rating = rec
    weight = WEIGHTS[etype]
    if rating:
        weight += rating # purchases with a rating boost engagement score further
    return ((user, item), weight)

# map -> reduceByKey: classic RDD pattern for aggregating engagement per (user,item)
engagement_rdd = events_rdd.map(to_weighted).reduceByKey(lambda a, b: a + b)

print("Aggregated (user,item) engagement scores:")
for row in sorted(engagement_rdd.collect()):
    print(" ", row)

# ---- Step 3: Utility-Based Agent - score & rank candidate items per user ----
# Utility combines: engagement score (relevance) + price-fit (prefer mid-range) + novelty (not already
# purchased)
purchased_items = set(events_rdd.filter(lambda r: r[2] == "purchase").map(lambda r: (r[0],
r[1])).collect())
price_lookup = dict(events_rdd.map(lambda r: (r[1], r[3])).distinct().collect())

def price_fit(price):
    # simple utility term: items priced 20-45 get the best price-fit score
    return 1.0 if 20 <= price <= 45 else 0.6

def utility(user, item, engagement):
    novelty = 0.0 if (user, item) in purchased_items else 1.0
    return 0.6 * min(engagement / 10.0, 1.0) + 0.25 * price_fit(price_lookup[item]) + 0.15 * novelty

scored_rdd = engagement_rdd.map(lambda kv: (kv[0][0], kv[0][1], utility(kv[0][0], kv[0][1], kv[1])))

# groupBy user -> sort by utility desc -> top-2 recommendations per user
recs_rdd = (
    scored_rdd
    .map(lambda r: (r[0], (r[1], r[2])))
    .groupByKey()
    .mapValues(lambda items: sorted(items, key=lambda x: -x[1])[:2])
)

print("\nTop-2 personalized recommendations per user (Utility-Based Agent output):")
for user, items in sorted(recs_rdd.collect()):
    print(f" {user}: {items}")

spark.stop()

```

Verified console output (this script was executed in this environment):

```

Aggregated (user,item) engagement scores:
(('U1', 'I100'), 3.0)
(('U1', 'I101'), 10.0)
(('U1', 'I102'), 1.0)
(('U1', 'I104'), 2.0)
(('U2', 'I100'), 9.0)
(('U2', 'I103'), 3.0)
(('U3', 'I101'), 1.0)
(('U3', 'I104'), 8.0)

```

Top-2 personalized recommendations per user (Utility-Based Agent output):

```
U1: [('I101', 0.85), ('I100', 0.58)]
U2: [('I100', 0.79), ('I103', 0.48)]
U3: [('I104', 0.73), ('I101', 0.46)]
```

Interpretation: U1's highest-engagement item (I101, a purchase) tops their utility ranking; U3's own purchase (I104) likewise dominates despite a lower raw engagement score than might be expected, because the utility function's price-fit and novelty terms reward it — exactly the multi-factor trade-off a real recommender's ranking stage performs.

2. Reverse Engineering a Smart Traffic Management System

***Scenario:** A city-wide traffic system adapts signals in real time based on vehicle density and congestion patterns. Reverse engineer the underlying AI system by describing the data pipeline using RDD-based streaming, identifying agent interactions, and explaining how decisions are made under time constraints.*

Reverse-Engineered Architecture

Observed behaviour: signal timing changes within seconds of a change in traffic density, responds instantly to emergency vehicles, and each junction appears to make its own decision independently of city-wide coordination. This points to:

- **Data Collection:** per-junction sensors (induction loops, cameras) emit vehicle-count, pedestrian-button, and emergency-vehicle-flag readings continuously onto a per-junction streaming topic.
- **Data Pipeline (RDD-based streaming):** Spark Structured Streaming ingests each short time-window as a micro-batch RDD; a map groups raw per-approach readings into a single per-junction percept via `reduceByKey`, exactly recreating the 'windowed aggregation' needed before any decision can be made.
- **Agent Interactions:** each junction runs its own independent Reactive Agent instance — there is no central controller in the loop for routine decisions (that would be too slow), only a shared streaming backbone that could optionally let neighbouring junctions exchange congestion state for coordinated 'green wave' timing.
- **Decisions Under Time Constraints:** because the agent is purely reactive (condition-action rules evaluated fresh on every micro-batch, no lookahead search), the entire decide-and-act loop completes well within the sub-second budget real-time signal control requires; a strict rule-priority order (fail-safe > emergency > pedestrian > congestion) keeps the decision deterministic even under this time pressure.

Implementation (PySpark / RDD)

system2_traffic.py

```
"""
System 2 - Reverse-Engineered Smart Traffic Management System
-----
Reconstructs a city-wide adaptive traffic signal pipeline: streaming vehicle-
density readings per junction -> RDD-based windowed aggregation -> a Reactive
Agent per junction that decides signal phase under a strict real-time budget.

Run: python3 system2_traffic.py
"""
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReverseEng-TrafficManagement").master("local[2]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# ---- Step 1: Simulated micro-batch of streaming sensor readings ----
# In production this is a Spark Structured Streaming DataFrame reading from a
# Kafka topic per junction, refreshed every 1-2 seconds. Here we simulate one
# micro-batch as an RDD: (junction_id, approach, vehicle_count, ped_waiting, emergency_flag, timestamp)
sensor_batch = [
    ("J1", "North", 14, False, False, 100),
    ("J1", "South", 4, True, False, 100),
    ("J1", "East", 2, False, False, 100),
    ("J1", "West", 3, False, False, 100),
```

```

    ("J2", "North", 5, False, False, 100),
    ("J2", "South", 6, False, True, 100),    # emergency vehicle approaching
    ("J3", "North", 2, False, False, 100),
    ("J3", "South", 2, False, False, 100),
]
sensor_rdd = sc.parallelize(sensor_batch)

# ---- Step 2: RDD transformations - windowed feature extraction per junction ----
# groupBy junction to build each junction's local percept (this recreates the
# "windowed aggregation" a Structured Streaming job performs every micro-batch)
def to_junction_kv(rec):
    junction, approach, count, ped, emergency, ts = rec
    return (junction, [(approach, count, ped, emergency)])

junction_percepts = sensor_rdd.map(to_junction_kv).reduceByKey(lambda a, b: a + b)

# ---- Step 3: Reactive Agent - condition-action rule evaluation (no memory, no lookahead) ----
def reactive_agent(junction, approaches):
    # Rule 1 (highest priority): emergency vehicle -> force green on that approach
    for appr, count, ped, emergency in approaches:
        if emergency:
            return (junction, f"EMERGENCY OVERRIDE: force GREEN on {appr}, RED elsewhere")

    # Rule 2: pedestrian waiting AND all vehicle queues light -> serve pedestrians
    ped_waiting = [a for a in approaches if a[2]]
    if ped_waiting and all(c <= 5 for _, c, _, _ in approaches):
        return (junction, f"PEDESTRIAN PHASE inserted for {ped_waiting[0][0]}")

    # Rule 3: congestion relief -> extend green on the most congested approach
    busiest = max(approaches, key=lambda a: a[1])
    if busiest[1] > 10:
        return (junction, f"EXTEND GREEN on {busiest[0]} (queue={busiest[1]})")

    # Rule 4 (default / fail-safe): fixed-time cycle
    return (junction, "DEFAULT fixed-time cycle (all approaches light)")

decisions_rdd = junction_percepts.map(lambda kv: reactive_agent(kv[0], kv[1]))

print("Reactive Agent signal decisions for this streaming micro-batch:")
for junction, decision in sorted(decisions_rdd.collect()):
    print(f" {junction}: {decision}")

spark.stop()

```

Verified console output (this script was executed in this environment):

```

Reactive Agent signal decisions for this streaming micro-batch:
J1: EXTEND GREEN on North (queue=14)
J2: EMERGENCY OVERRIDE: force GREEN on South, RED elsewhere
J3: DEFAULT fixed-time cycle (all approaches light)

```

Interpretation: J1's North approach queue (14 vehicles) triggers the congestion-relief rule; J2's emergency-vehicle flag overrides every other rule immediately; J3, with light traffic on all approaches, correctly falls back to the safe default — demonstrating the exact rule-priority behaviour inferred above.

3. Reverse Engineering a Fraud Detection Pipeline

Scenario: A banking system flags fraudulent transactions within milliseconds. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how feature extraction, anomaly detection, and agent-based alert mechanisms are likely implemented.

Reverse-Engineered Architecture

Observed behaviour: transactions are approved or blocked almost instantly; blocks correlate with unusually large amounts, foreign countries the account rarely uses, and multiple transactions in rapid succession. Reverse-engineering this points to:

- Data Collection: every card-swipe/transfer event streams in as (txn_id, account_id, amount, country, timestamp), alongside a separately-maintained per-account 'normal behaviour' profile (average spend, home country) refreshed by a periodic batch job.
- Feature Extraction (RDD transformations): a map/reduceByKey groups transactions per account, sorts them by time, and computes three signal features per transaction — deviation from the account's average amount, time-gap ('velocity') since the previous transaction, and a geo-mismatch flag against the home country.
- Anomaly Detection: a weighted scoring function combines these three features into a single anomaly score (large deviation, geo-mismatch, and high velocity each add to the score) — this is why a large, foreign, rapid-fire transaction gets flagged while a normal local purchase does not.
- Agent-Based Alert Mechanism: a Goal-Based Agent's goal is to reach a confident fraud/not-fraud determination; transactions scoring below a low threshold are approved (goal: 'fraud ruled out' reached), mid-range scores trigger a step-up authentication request (goal not yet resolved — agent seeks more evidence), and high scores trigger an immediate block-and-alert action (goal: 'fraud confirmed' reached) — explaining the millisecond decisiveness without needing a human in the loop for clear-cut cases.

Implementation (PySpark / RDD)

system3_fraud.py

```
"""
System 3 - Reverse-Engineered Fraud Detection Pipeline
-----
Reconstructs a millisecond-latency banking fraud pipeline: raw transaction
stream -> RDD-based feature extraction (velocity, deviation-from-normal,
geo-mismatch) -> anomaly-scoring -> a Goal-Based Agent whose goal is
"confirm/deny fraud with high confidence" and that fires an alert action.

Run: python3 system3_fraud.py
"""
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReverseEng-FraudDetection").master("local[2]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# ---- Step 1: Raw transaction stream ----
# (txn_id, account_id, amount, country, timestamp_seconds)
transactions = [
    ("T1", "A1", 45.0, "IN", 1000),
    ("T2", "A1", 60.0, "IN", 1005),
    ("T3", "A1", 4200.0, "RU", 1006), # huge amount, foreign country, seconds after T2 -> suspicious
    ("T4", "A2", 120.0, "IN", 2000),
    ("T5", "A2", 130.0, "IN", 2500),
    ("T6", "A3", 80.0, "IN", 3000),
    ("T7", "A3", 5000.0, "US", 3002), # huge amount + foreign + near-instant -> suspicious
]
txn_rdd = sc.parallelize(transactions)

# ---- Step 2: Feature extraction via RDD transformations ----
# Historical "normal" behaviour per account (in production this comes from a
# pre-aggregated profile table refreshed by a separate Spark batch job).
account_profile = {"A1": {"avg_amount": 55.0, "home_country": "IN"},
                   "A2": {"avg_amount": 125.0, "home_country": "IN"},
                   "A3": {"avg_amount": 80.0, "home_country": "IN"}}

# groupBy account -> sort by time -> compute velocity (seconds since previous txn)
def key_by_account(rec):
    txn_id, acct, amount, country, ts = rec
    return (acct, [(txn_id, amount, country, ts)])

grouped = txn_rdd.map(key_by_account).reduceByKey(lambda a, b: a + b)

def extract_features(acct, txns):
    txns_sorted = sorted(txns, key=lambda t: t[3])
    profile = account_profile[acct]
```

```

features = []
prev_ts = None
for txn_id, amount, country, ts in txns_sorted:
    velocity_gap = (ts - prev_ts) if prev_ts is not None else None
    deviation = amount / profile["avg_amount"]
    geo_mismatch = country != profile["home_country"]
    features.append((txn_id, acct, amount, country, deviation, velocity_gap, geo_mismatch))
    prev_ts = ts
return features

features_rdd = grouped.flatMap(lambda kv: extract_features(kv[0], kv[1]))

print("Extracted per-transaction fraud features (deviation, velocity_gap, geo_mismatch):")
for row in features_rdd.collect():
    print(" ", row)

# ---- Step 3: Anomaly scoring + Goal-Based Agent decision ----
# Goal: reach a confident fraud/not-fraud determination and take the matching action.
def anomaly_score(feats):
    txn_id, acct, amount, country, deviation, velocity_gap, geo_mismatch = feats
    score = 0.0
    if deviation > 5:
        score += 0.5
    if geo_mismatch:
        score += 0.3
    if velocity_gap is not None and velocity_gap < 10:
        score += 0.3
    return (txn_id, acct, round(score, 2))

def goal_based_decision(scored):
    txn_id, acct, score = scored
    if score >= 0.8:
        return (txn_id, acct, score, "BLOCK + ALERT (fraud goal confirmed)")
    elif score >= 0.4:
        return (txn_id, acct, score, "STEP-UP AUTH requested (goal uncertain - gather more evidence)")
    else:
        return (txn_id, acct, score, "APPROVE (fraud goal ruled out)")

decisions_rdd = features_rdd.map(anomaly_score).map(goal_based_decision)

print("\nGoal-Based Agent fraud decisions:")
for txn_id, acct, score, decision in decisions_rdd.collect():
    print(f" {txn_id} ({acct}) score={score}: {decision}")

spark.stop()

```

Verified console output (this script was executed in this environment):

```

Extracted per-transaction fraud features (deviation, velocity_gap, geo_mismatch):
('T1', 'A1', 45.0, 'IN', 0.82, None, False)
('T2', 'A1', 60.0, 'IN', 1.09, 5, False)
('T3', 'A1', 4200.0, 'RU', 76.36, 1, True)
('T4', 'A2', 120.0, 'IN', 0.96, None, False)
('T5', 'A2', 130.0, 'IN', 1.04, 500, False)
('T6', 'A3', 80.0, 'IN', 1.0, None, False)
('T7', 'A3', 5000.0, 'US', 62.5, 2, True)

Goal-Based Agent fraud decisions:
T1 (A1) score=0.0: APPROVE (fraud goal ruled out)
T2 (A1) score=0.3: APPROVE (fraud goal ruled out)
T3 (A1) score=1.1: BLOCK + ALERT (fraud goal confirmed)
T4 (A2) score=0.0: APPROVE (fraud goal ruled out)
T5 (A2) score=0.0: APPROVE (fraud goal ruled out)
T6 (A3) score=0.0: APPROVE (fraud goal ruled out)
T7 (A3) score=1.1: BLOCK + ALERT (fraud goal confirmed)

```

Interpretation: T3 and T7 — the two transactions with a huge amount, a foreign country, and a near-instant time-gap from the account's previous transaction — are the only ones blocked, exactly matching the fraud pattern inferred from the scenario's observed behaviour.

4. Reverse Engineering a Personalized Learning System

Scenario: An online learning platform adapts content difficulty based on student performance and engagement. Reverse engineer the system design by identifying how learner data is processed using RDD transformations, and explain how intelligent agents contribute to adaptive learning and feedback mechanisms.

Reverse-Engineered Architecture

Observed behaviour: two students starting at the same difficulty level diverge in the difficulty of items they are shown, based only on how they answered — correctness, hint usage, and speed all seem to matter, not just right/wrong. This implies:

- Data Collection: each answer submission is logged as (student_id, skill, item_difficulty, correct, response_time, hints_used, session_order) and streamed to the processing layer.
- RDD Transformations: a map/reduceByKey groups the interaction log by (student, skill) and sorts each group by session order — the sequential, per-skill grouping is essential, since the agent's mastery estimate must be updated one interaction at a time, in order.
- Intelligent Agent Contribution — the classic four-part Learning Agent: a Critic scores each outcome (a hint-free correct answer scores higher than a correct-with-hints answer, which scores higher than an incorrect one); a Learning Element uses that score to nudge a running mastery estimate up or down; and a Performance Element reads the updated mastery estimate to pick the next item's difficulty — this explains why hint-free, fast correct answers accelerate difficulty increases faster than a hesitant correct answer does.
- Feedback Mechanism: because mastery is recomputed after every single interaction (not in a nightly batch), the platform can adapt difficulty within the same session, matching the observed real-time responsiveness.

Implementation (PySpark / RDD)

system4_education.py

```
"""
System 4 - Reverse-Engineered Personalized Learning System
-----
Reconstructs an adaptive-difficulty education pipeline: raw learner
interaction logs -> RDD transformations computing per-skill mastery -> a
Learning Agent (Performance Element + Critic + Learning Element) that adapts
the next item's difficulty per learner.

Run: python3 system4_education.py
"""
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReverseEng-PersonalizedLearning").master("local[2]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# ---- Step 1: Raw interaction log stream ----
# (student_id, skill, item_difficulty[1-5], correct, response_time_sec, hints_used, session_order)
interactions = [
    ("S1", "Algebra", 3, True, 12, 0, 1),
    ("S1", "Algebra", 4, False, 30, 1, 2),
    ("S1", "Algebra", 3, True, 15, 1, 3),
    ("S1", "Algebra", 4, True, 10, 0, 4),
    ("S2", "Algebra", 3, True, 8, 0, 1),
    ("S2", "Algebra", 4, True, 9, 0, 2),
    ("S2", "Algebra", 5, True, 11, 0, 3),
]
log_rdd = sc.parallelize(interactions)

# ---- Step 2: RDD transformations - per (student, skill) sequential feature build ----
def key_by_student_skill(rec):
    student, skill, diff, correct, rtime, hints, order = rec
    return ((student, skill), [(diff, correct, rtime, hints, order)])

grouped = log_rdd.map(key_by_student_skill).reduceByKey(lambda a, b: a + b)
```



```
# ---- Step 3: Learning Agent - Performance Element / Critic / Learning Element loop ----
def learning_agent(key, records):
    student, skill = key
    records_sorted = sorted(records, key=lambda r: r[4]) # order by session_order
    mastery = 2.5 # initial baseline skill estimate (out of 5)
    trace = []
    for diff, correct, rtime, hints, order in records_sorted:
        # Critic: score this outcome against the performance standard
        outcome_quality = 1.0 if (correct and hints == 0) else (0.6 if correct else -0.5)
        # Learning Element: update mastery estimate (simple bounded update rule)
        mastery = max(1.0, min(5.0, mastery + 0.4 * outcome_quality))
        # Performance Element: choose next item's difficulty from updated mastery
        next_difficulty = round(mastery)
        trace.append({
            "session": order, "item_difficulty": diff, "correct": correct,
            "hints": hints, "updated_mastery": round(mastery, 2),
            "next_item_difficulty": next_difficulty,
        })
    return (student, skill, trace)

results_rdd = grouped.map(lambda kv: learning_agent(kv[0], kv[1]))

print("Learning Agent adaptive-difficulty trace per student/skill:")
for student, skill, trace in results_rdd.collect():
    print(f"\n Student {student} - Skill: {skill}")
    for step in trace:
        print("    ", step)

spark.stop()
```

Verified console output (this script was executed in this environment):

```
Learning Agent adaptive-difficulty trace per student/skill:

Student S1 - Skill: Algebra
  {'session': 1, 'item_difficulty': 3, 'correct': True, 'hints': 0, 'updated_mastery': 2.9,
'next_item_difficulty': 3}
  {'session': 2, 'item_difficulty': 4, 'correct': False, 'hints': 1, 'updated_mastery': 2.7,
'next_item_difficulty': 3}
  {'session': 3, 'item_difficulty': 3, 'correct': True, 'hints': 1, 'updated_mastery': 2.94,
'next_item_difficulty': 3}
  {'session': 4, 'item_difficulty': 4, 'correct': True, 'hints': 0, 'updated_mastery': 3.34,
'next_item_difficulty': 3}

Student S2 - Skill: Algebra
  {'session': 1, 'item_difficulty': 3, 'correct': True, 'hints': 0, 'updated_mastery': 2.9,
'next_item_difficulty': 3}
  {'session': 2, 'item_difficulty': 4, 'correct': True, 'hints': 0, 'updated_mastery': 3.3,
'next_item_difficulty': 4}
  {'session': 3, 'item_difficulty': 5, 'correct': True, 'hints': 0, 'updated_mastery': 3.7,
'next_item_difficulty': 4}
```

Interpretation: S2, who answers every item correctly without hints, is advanced to difficulty-4 and then difficulty-5 items, while S1 — who makes a mistake and needs hints along the way — is held around difficulty-3, exactly the divergent adaptive behaviour observed in the platform.

5. Reverse Engineering a Cloud-Based AI Monitoring System

Scenario: A cloud platform automatically scales resources and detects system anomalies in real time. Reverse engineer the architecture by analyzing how PySpark processes large-scale logs, how RDD operations support fault tolerance, and how intelligent agents coordinate monitoring, scaling, and anomaly response.

Reverse-Engineered Architecture

Observed behaviour: the platform adds/removes server capacity within moments of a load change, and separately raises incident alerts specifically when error rates spike (not just when load is high) — two distinct response types firing on different triggers. This suggests:

- Data Collection: every server continuously emits (server_id, region, cpu_pct, mem_pct, error_rate, timestamp) metric records onto a log-streaming layer, at large scale across many regions.
- Large-Scale Log Processing (PySpark): incoming metrics are loaded as an RDD automatically split into partitions across the Spark cluster's worker nodes; a map/reduceByKey groups records by region and computes rolling averages (CPU, memory) and worst-case error rate per region — this parallel partitioning is what lets the platform process metrics from potentially thousands of servers within a real-time budget.
- Fault Tolerance via RDDs: because each RDD partition's transformation lineage is tracked, if a worker node processing one region's partition fails mid-computation, Spark can recompute just that lost partition from the original log data rather than losing the whole job or corrupting other regions' results — this is precisely why a single-node failure in the observed platform never seems to cause monitoring blackouts elsewhere.
- Agent Coordination: a Multi-Agent System splits responsibility — Monitoring Agents (the per-region aggregation step) continuously produce summaries; a Scaling Agent reads each region's averaged load and independently decides to scale out, scale in, or hold; and a separate Anomaly-Response Agent inspects individual server records for error-rate spikes and fires alerts/auto-restarts — explaining why scaling and anomaly alerts are clearly two independent decision processes rather than one combined rule.

Implementation (PySpark / RDD)

system5_cloud_monitoring.py

```
"""
System 5 - Reverse-Engineered Cloud-Based AI Monitoring System
-----
Reconstructs an auto-scaling / anomaly-detecting cloud monitoring pipeline:
raw per-server metric logs -> RDD-based fault-tolerant aggregation -> a
Multi-Agent System (Monitoring Agents + Scaling Agent + Anomaly-Response
Agent) that coordinates scaling and incident response decisions.

Run: python3 system5_cloud_monitoring.py
"""
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ReverseEng-CloudMonitoring").master("local[2]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")

# ---- Step 1: Raw per-server metric log stream ----
# (server_id, region, cpu_pct, mem_pct, error_rate_pct, timestamp)
metrics = [
    ("srv-1", "ap-south", 45, 50, 0.1, 100),
    ("srv-2", "ap-south", 92, 88, 0.2, 100),
    ("srv-3", "ap-south", 40, 45, 0.1, 100),
    ("srv-4", "us-east", 95, 91, 6.5, 100),    # high load AND high error rate -> anomaly
    ("srv-5", "us-east", 30, 40, 0.0, 100),
    ("srv-6", "eu-west", 55, 60, 0.1, 100),
]
metrics_rdd = sc.parallelize(metrics)

# RDD is partitioned/replicated across the cluster automatically by Spark,
# which is what gives this pipeline fault tolerance: if a worker node fails,
# lost partitions are recomputed from lineage rather than losing data.
print(f"Metrics RDD partitions (fault-tolerance unit): {metrics_rdd.getNumPartitions()}")

# ---- Step 2: Monitoring Agents - per-region aggregation via RDD transformations ----
def key_by_region(rec):
    server, region, cpu, mem, err, ts = rec
    return (region, [(server, cpu, mem, err)])

region_grouped = metrics_rdd.map(key_by_region).reduceByKey(lambda a, b: a + b)

def region_summary(region, servers):
```

```

    avg_cpu = sum(s[1] for s in servers) / len(servers)
    avg_mem = sum(s[2] for s in servers) / len(servers)
    max_err = max(s[3] for s in servers)
    return (region, round(avg_cpu, 1), round(avg_mem, 1), max_err, servers)

summaries_rdd = region_grouped.map(lambda kv: region_summary(kv[0], kv[1]))

print("\nMonitoring Agent per-region summaries (avg_cpu, avg_mem, max_error_rate):")
for region, avg_cpu, avg_mem, max_err, servers in summaries_rdd.collect():
    print(f"  {region}: avg_cpu={avg_cpu}%, avg_mem={avg_mem}%, max_error_rate={max_err}%")

# ---- Step 3: Multi-Agent coordination - Scaling Agent + Anomaly-Response Agent ----
def scaling_agent(region, avg_cpu, avg_mem):
    if avg_cpu > 80 or avg_mem > 80:
        return f"SCALE OUT (+2 instances) in {region} (avg_cpu={avg_cpu}%, avg_mem={avg_mem}%)"
    elif avg_cpu < 35 and avg_mem < 35:
        return f"SCALE IN (-1 instance) in {region} (avg_cpu={avg_cpu}%, avg_mem={avg_mem}%)"
    return f"HOLD capacity in {region}"

def anomaly_response_agent(region, servers):
    alerts = []
    for server, cpu, mem, err in servers:
        if err > 5.0:
            alerts.append(f"ANOMALY ALERT: {server} in {region} error_rate={err}% -> auto-restart + page on-call")
    return alerts if alerts else [f"No anomalies detected in {region}"]

print("\nScaling Agent decisions:")
for region, avg_cpu, avg_mem, max_err, servers in summaries_rdd.collect():
    print("  ", scaling_agent(region, avg_cpu, avg_mem))

print("\nAnomaly-Response Agent actions:")
for region, avg_cpu, avg_mem, max_err, servers in summaries_rdd.collect():
    for alert in anomaly_response_agent(region, servers):
        print("  ", alert)

spark.stop()

```

Verified console output (this script was executed in this environment):

```

Metrics RDD partitions (fault-tolerance unit): 2

Monitoring Agent per-region summaries (avg_cpu, avg_mem, max_error_rate):
eu-west: avg_cpu=55.0%, avg_mem=60.0%, max_error_rate=0.1%
ap-south: avg_cpu=59.0%, avg_mem=61.0%, max_error_rate=0.2%
us-east: avg_cpu=62.5%, avg_mem=65.5%, max_error_rate=6.5%

Scaling Agent decisions:
  HOLD capacity in eu-west
  HOLD capacity in ap-south
  HOLD capacity in us-east

Anomaly-Response Agent actions:
  No anomalies detected in eu-west
  No anomalies detected in ap-south
  ANOMALY ALERT: srv-4 in us-east error_rate=6.5% -> auto-restart + page on-call

```

Interpretation: even though us-east's average CPU/memory is not high enough to trigger scaling, the Anomaly-Response Agent independently catches srv-4's error-rate spike and raises an alert — confirming that scaling decisions and anomaly detection are handled by two separate, specialized agents rather than a single combined rule, exactly as reverse-engineered from the observed platform behaviour.

Cross-System Summary

System	Key RDD Pattern Used	Intelligent Agent Type	Core Decision Trigger
1. Recommendation System	map -> reduceByKey (engagement scoring)	Utility-Based Agent	Highest combined utility score (relevance + price-fit + novelty)
2. Traffic Management	map -> reduceByKey (windowed per-junction percept)	Reactive Agent	Condition-action rule match on current sensor percept
3. Fraud Detection	map -> reduceByKey -> flatMap (feature extraction)	Goal-Based Agent	Anomaly score crossing a confidence threshold
4. Personalized Learning	map -> reduceByKey (sequential per-skill grouping)	Learning Agent (Critic + Learning + Performance elements)	Updated mastery estimate after each interaction
5. Cloud AI Monitoring	map -> reduceByKey (regional aggregation, partitioned RDD)	Multi-Agent System (Monitoring + Scaling + Anomaly-Response)	Load threshold (scaling) or error-rate spike (anomaly alert)